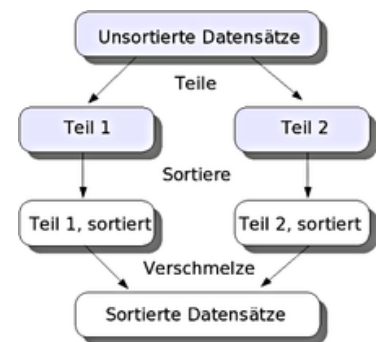


Praktikum 2: Merge Sort

Ein bisher nicht betrachtetes Sortierverfahren ist der sog. Merge-Sort. Es handelt sich dabei wieder um einen "Teile-und-Herrsche"-Algorithmus. Die Grundidee ist dabei folgende:

- I. Um ein Array zu sortieren betrachtet man zunächst die Länge des Arrays. Hat es die Länge 1, ist es sortiert und wir sind fertig (Basisfall).
- II. Andernfalls teilen wir das Array in zwei möglichst gleich große Teilarrays (links und rechts) und rufen das Sortierverfahren für jedes erneut rekursiv auf. Anschließend müssen die beiden sortierten Teilarrays zusammengeführt werden ("Merge").



Der entscheidende Schritt des Verfahrens ist das **Merge**

(Verschmelzen): Hierbei werden die zwei bereits sortierten Teilarrays mithilfe von zwei Zeigern (Indices) gleichzeitig durchlaufen. In jedem

Schritt wird das jeweils kleinere Element der beiden Zeigerpositionen ausgewählt und in ein Hilfsarray übertragen, woraufhin der entsprechende Zeiger eine Position weiterwandert. Dieser Prozess wird so lange fortgesetzt, bis eines der Teilarrays vollständig geleert ist. Abschließend werden die verbleibenden Elemente des noch nicht erschöpften Arrays direkt an das Ende des Hilfsarrays angehängt, wodurch ein zusammenhängend sortierter Datensatz entsteht.

1. Analysieren Sie den Algorithmus zunächst im Hinblick auf die Eigenschaften Stabilität, In-Place und Laufzeit. Mit wie vielen Vergleichen rechnen Sie in Abhängigkeit von n ?
2. Implementieren Sie den beschriebenen Algorithmus und sortieren Sie die vorgegebenen Folgen vom letzten Praktikum (seq0.txt bis seq3.txt). Messen Sie die Laufzeit des Sortiervorgangs und ermitteln Sie die Anzahl der Vergleiche mit Hilfe der `utils`-Klassen. Deckt sich Ihr Messergebnis mit Ihren Erwartungen?
3. In der Vorlesung haben wir gelernt, dass die Korrektheit oft über eine Schleifeninvariante nachgewiesen wird. Betrachten Sie Ihren Code für den Merge-Schritt (das Zusammenführen zweier Teilarrays).
 - a) Formulieren Sie eine Invariante für die Schleife, welche die beiden Teilarrays zu einem Hilfsarray zusammenfügt. Hinweis: Betrachten Sie den Zustand des Zielarrays im Verhältnis zu den bereits gelesenen Elementen der Quellarrays.
 - b) Zeigen Sie kurz auf, warum diese Invariante nach der Initialisierung vor dem ersten Vergleich wahr ist.
 - c) Begründen Sie, warum die Invariante bei Terminierung die vollständige Sortierung des Teilabschnitts garantiert.
4. Hier reicht die Schleifeninvariante für den Nachweis der Korrektheit nicht aus. Wir benötigen noch einen weiteren Induktionsbeweis für die Rekursion (**Strukturelle Induktion**). Wie könnte der informell aussehen? (Hinweis: Denken Sie an den Basisfall und die Annahme, dass die rekursiven Aufrufe für kleinere Teilarrays bereits korrekt funktionieren.)
5. Sind hier Unterschiede bei einer Worst Case/Best Case-Betrachtung zu erwarten? Begründen Sie Ihre Antwort kurz mit Blick auf die Struktur des Algorithmus.

Musterlösung Praktikum 2: Merge Sort

1 Analyse des Algorithmus

1.1 Informelle Analyse

Basierend auf der Funktionsweise von Merge Sort lassen sich folgende Eigenschaften ableiten:

- **Stabilität:** Merge Sort ist **stabil**. Da bei der Verschmelzung (*Merge*) im Falle von Gleichheit zweier Elemente das Element aus dem linken Teilarray bevorzugt wird, bleibt die relative Reihenfolge identischer Elemente erhalten.
- **In-Place:** Der Algorithmus ist **nicht In-Place**. Er benötigt ein Hilfsarray, um die Elemente während des Verschmelzungsprozesses zwischenzuspeichern.
- **Laufzeit:** Die theoretische Laufzeit liegt bei $O(n \cdot \log n)$. Dies liegt daran, dass das Array immer halbiert wird ($\log n$ Ebenen) und auf jeder Ebene $O(n)$ Aufwand für das Verschmelzen entsteht. In Abhängigkeit von n ist also mit etwa $n \cdot \log_2 n$ Vergleichen zu rechnen.

1.2 Laufzeitanalyse via Master-Theorem

Die Laufzeit von Merge Sort lässt sich über eine Rekursionsgleichung beschreiben. Da das Array in jedem Schritt in zwei etwa gleich große Teilarrays zerlegt wird und der Aufwand für das Verschmelzen (Merge-Schritt) linear zur Anzahl der Elemente n ist, ergibt sich:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + f(n) \quad \text{mit} \quad f(n) = \Theta(n)$$

Zur Lösung verwenden wir das **Master-Theorem** für Rekursionen der Form

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

:

1. Parameter bestimmen:

- $a = 2$: Anzahl der rekursiven Aufrufe pro Ebene.
- $b = 2$: Faktor, um den die Problemgröße reduziert wird.
- $f(n) = \Theta(n^1)$: Der Aufwand für den Divide-and-Conquer-Schritt (das Mergen).

2. **Vergleich mit dem kritischen Exponenten:** Wir berechnen

$$\log_b(a) = \log_2(2) = 1$$

3. **Anwendung des Master-Theorems:** Da $f(n) = \Theta(n^{\log_b(a)}) = \Theta(n^1)$, greift der **zweite Fall** des Master-Theorems.

4. **Ergebnis:** Daraus folgt direkt die Gesamtlaufzeit:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(n \cdot \log n)$$

Dies bestätigt die theoretische Laufzeit von $O(n \log n)$ für alle Fälle.

2 Schleifeninvariante für den Merge-Schritt

2.1 Formulierung der Invariante

Für die Schleife, die zwei sortierte Teilarrays in ein Hilfsarray zusammenfügt, lässt sich folgende Invariante formulieren: *Zu Beginn jeder Iteration enthält das Hilfsarray $B[1 \dots k]$ die kleinsten k Elemente der beiden Teilarrays L und R in sortierter Reihenfolge, wobei k die Anzahl der bereits gelesenen Elemente ist.*

2.2 Nachweis

- **Initialisierung:** Vor dem ersten Vergleich ist das Hilfsarray leer ($k = 0$). Die Invariante ist trivialerweise wahr, da ein leeres Array sortiert ist.
- **Invarianz (Erhaltung):** Angenommen, die Invariante gilt für k . Das Hilfsarray enthält die k kleinsten Elemente von $L \cup R$ sortiert. Es seien i und j die aktuellen Indizes in den Teilarrays L und R .
 - Im Schritt $k \rightarrow k + 1$ wird das kleinere der beiden Elemente $L[i]$ und $R[j]$ gewählt.
 - Da L und R sortiert sind, ist das gewählte Element (z. B. $L[i]$) kleiner oder gleich allen verbleibenden Elementen in $L[i \dots]$ und $R[j \dots]$.
 - Da die Invariante für k galt, sind alle Elemente in $B[1 \dots k]$ bereits kleiner oder gleich $L[i]$.
 - Das Anfügen von $L[i]$ an Position $B[k + 1]$ erhält somit sowohl die Sortierung als auch die Eigenschaft, dass $B[1 \dots k + 1]$ die $k + 1$ kleinsten Elemente enthält.
- **Terminierung:** Bei Terminierung ist eines der Teilarrays vollständig geleert. Die verbleibenden Elemente des anderen Arrays werden angehängt. Da die Teilarrays selbst sortiert waren und nur Elemente angehängt werden, die größer oder gleich den bereits im Hilfsarray befindlichen Elementen sind, ist der gesamte Teilabschnitt am Ende vollständig sortiert.

3 Strukturelle Induktion (Rekursion)

Ein informeller Beweis für die Korrektheit der Rekursion erfolgt über strukturelle Induktion:

- **Basisfall:** Ein Array der Länge 1 ist per Definition sortiert.
- **Induktionsschritt:** Wir nehmen an, dass der rekursive Aufruf für Arrays der Größe $< n$ korrekt funktioniert (Induktionsannahme). Da Merge Sort das Array in zwei kleinere Teilarrays teilt, liefert die Rekursion zwei korrekt sortierte Teilfolgen zurück. Da der *Merge*-Schritt diese zwei sortierten Folgen nachweislich korrekt verschmilzt, ist auch das Gesamtarray der Größe n korrekt sortiert.

4 Best Case / Worst Case Betrachtung

- **Erwartung:** Es sind **keine wesentlichen Unterschiede** in der asymptotischen Laufzeit ($O(n \log n)$) zu erwarten.
- **Begründung:** Die Struktur des Algorithmus erzwingt unabhängig von der Anordnung der Daten immer dieselbe Anzahl an Teilungsschritten (rekursive Aufrufe). Auch der Verschmelzungsprozess muss immer alle Elemente durchlaufen, bis mindestens ein Teilarray leer ist, was die Komplexität stabil hält.